

# Drone Health Mapping Pipeline

## Project Documentation

**Ramazan Batanov**  
Hof University  
Hof, Germany

**Berkay Gök**  
Hof University  
Hof, Germany

**Yevhenii Raietskyi**  
Hof University  
Hof, Germany

**Aseem Bhide**  
Hof University  
Hof, Germany

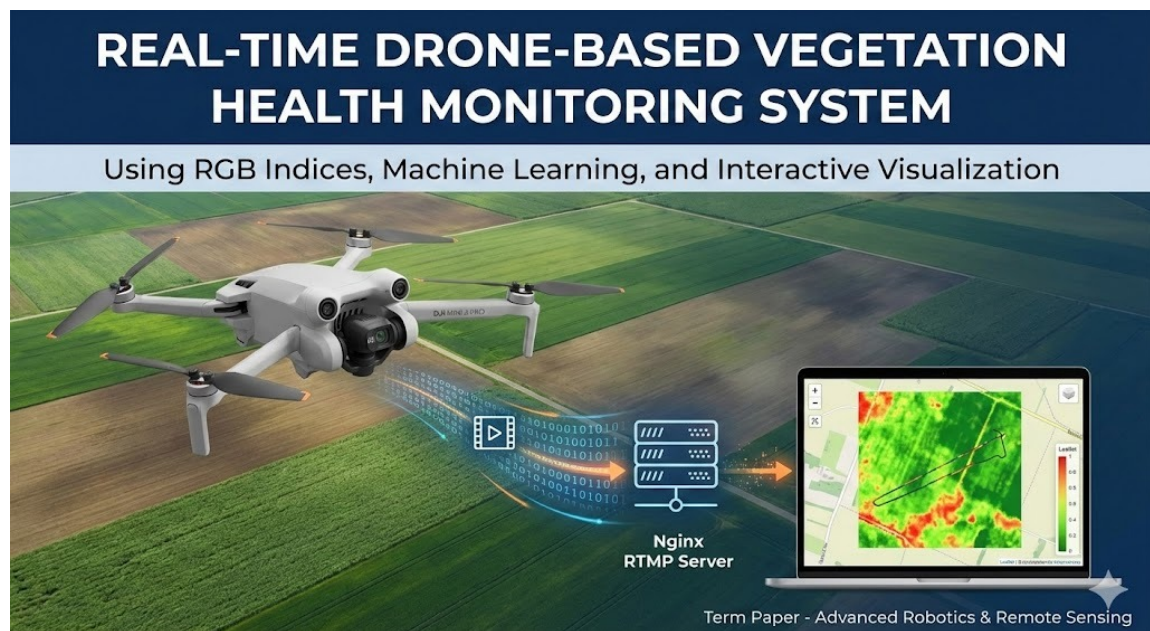
**Gehad Hamada**  
Hof University  
Hof, Germany

**Sultan Sailau**  
Hof University  
Hof, Germany

**Aditya Dalvi**  
Hof University  
Hof, Germany

**Matheus Hipolito Carvalho**  
Hof University  
Hof, Germany

**Abhishek Sondkar**  
Hof University  
Hof, Germany



#### ACM Reference Format:

. 2026. . In *Proceedings of Drone Health Mapping Conference (Termpaper '26)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## 1 Introduction

A great number of drones have been deployed in the last few years to perform tasks related to the monitoring of vegetation. The ability to collect data over a very wide area quickly represents one of the greatest benefits of this technology, which provides an additional aerial view point that may be difficult to achieve using land based observation methods. For this research project, the DJI Mini 4 Pro (figure 1, left) was selected as the main drone used to capture the vegetation footage [1]. Our goal went beyond merely flying and taking video of the drone's flight path; rather, we were looking to develop a comprehensive workflow that would connect acquisition, processing and visualization into one single piece of equipment[3]. To create a dataset that could be compared between multiple flights, one method is to create waypoint mission flights that eliminate pilot error while creating the same flight paths every time. With a resolution of 4K, the video captures will allow us to analyze each frame and extract features if needed. We are also interested in exploring the possibilities of real time monitoring. Instead of having to wait until the end of a flight for processing, we can now publish the live video feed online via RTMP. We have developed an architecture that allows the drone feed to send a feed to a RTMP server on an Nginx server that is then consumed by a Python client for down-stream processing. This allows us to provide a reliable, consistent streaming service that allows us to demonstrate the feasibility of real-time analytics, as well as expand our pipeline to include other uses of the video feed in real-time analytics. There are several features extracted from images that can be used to perform vegetation analysis. As we are currently limited to RGB cameras, we are currently using three RGB-based vegetation indices (VARI, excess Green and TGI) as they represent the greenness and health of the plants and do not require the use of multi-spectral sensors. We are using these features in a machine learning pipeline with an incremental classifier (SGDClassifier in logistic regression mode) to identify between healthy and unhealthy vegetation. For better interpretability, we have added an interactive map visualizer in Leaflet, where analytical overlays and flight tracks can be viewed in a web browser. Overall, the research has demonstrated a working student-level protocol illustrating the entire process of collecting data from a drone to analysis and visualization. Although the research revealed moderate performance levels, it also pointed out many areas of weakness including the effect of shadows on vegetation, varying light conditions, decreasing information quality at high altitudes and unstable image information from consecutive frames that indicate areas for future improvements[7].

## 2 Drone Specs and Data

The DJI Mini 4 Pro is a portable, lightweight drone that has been designed for use by all types of users, however due to its high quality

RGB camera and good flight stability as well as the navigation sensors built into the drone [1], it has many capabilities to allow it to be used for research-oriented uses such as collecting aerial imagery and creating datasets for vegetation monitoring and analysis. Due to the drones portability, it can easily be deployed in various environments.

Key features relevant to this study include:

- **RGB Imaging:** up to 48 MP still images and 4K video recording (up to 60 fps, depending on settings)
- **Stabilization:** mechanical gimbal stabilization for low-motion-blur video suitable for frame-level processing
- **Navigation:** integrated GNSS positioning (GPS, Galileo, BeiDou) for repeatable flight paths
- **Mission support:** compatibility with waypoint-style data collection workflows through DJI Fly (when supported by the mission mode and configuration)

Although the drone estimates position during flight, the availability of georeferenced telemetry in exported files depends on recording settings, firmware/app version, and the chosen data products. For this reason, telemetry availability is treated as a *validated input* rather than an assumption[7].

### 2.1 Video Link & Capture

In order to provide the necessary level of accuracy for aerial images and video footage, the aerial system features a 1/1.3-inch CMOS camera unit capable of taking photographs at 48 megapixels, as well as capturing high-quality video at resolutions of 4K Ultra High Definition (UHD) at frame rates of 60 frames per second (fps). Aerial photographs are captured using the O4 Long Range Transmission System, which features four antennas that enable it to transmit signals a maximum distance of 12 miles (20 kilometers) when there is a clear line of sight. Depending on the intended use, the aerial system is designed to allow for live video feeds or recorded video files to be accessed for advanced analytical purposes. For example, live video feeds are transmitted through the DJI Fly App or embedded display of the DJI RC 2 remote control for real time viewing. Some users have utilized the RTMP protocol via an Nginx and Docker-based server to transmit video streams from the aerial system into advanced robotics systems[6]. If no standalone wireless network exists, a mobile device can act only as an intermediary system for transmitting drone commands while connected to a wireless network. All recorded video files for advanced analytical uses (primarily vegetation health mapping) are stored in 4K resolution on a high speed UHS-I Speed Class 3 micro SD card to support the maximum 150 Mbps bitrate. The primary video format used is MP4, with either the megapixel video compression standard MPEG-4 AVC/H.264 or the HEVC/H.265 standard being utilized to encode video files. Additionally, geospatial information (latitude, longitude, relative altitude, etc.) are recorded as part of the video files in the SRT format, which can then be converted to a CSV format for later retrieval of the spatial and temporal information of each individual frame of video. Static images captured with the aerial system are recorded in the DNG (raw)/RAW format to ensure that all of the available spectral and dynamic information is captured for the purpose of calculating vegetation indices. Ensuring flight stability and maintaining the integrity of

*Termpaper '26, Hof, Germany*

2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM  
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

the data collected during autonomous or manually controlled flights are paramount in ensuring the accuracy of the data collected. The aerial system features a 312  $\frac{3}{4}$  degree  $\pm$  -90° to 60° controllable tilt pan to maintain stable imagery even when flying at high velocities. Positional stability is achieved using the combination of GNSS (Global Navigation Satellite Systems) technology (GPS, Galileo and BeiDou), and a 360-degree omnidirectional vision system that consists of four binocular cameras and a 3D infrared sensing system. In addition to these capabilities, specialized flight modes (i.e., Cine Mode) exist specifically for robotic applications where maintaining high horizontal velocities and achieving stable shooting conditions are paramount. However, technical limitations associated with longer braking distances at higher altitudes, and signal interference from power lines and base stations must be avoided so as to prevent "ATTI mode" transitions that result in horizontal drift.

## 2.2 Waypoints for Dataset Collection

Waypoints are predefined coordinates that guide the drone along a planned path. For this project, waypoint-style collection is used to increase repeatability and reduce manual piloting variability. By specifying target altitude, speed, and camera orientation, the same area can be recorded under comparable acquisition conditions across flights[2].

Using waypoint planning helps ensure:

- Even spatial coverage of vegetation areas
- Reduced variability between flights due to pilot behavior
- Fairer comparisons across different runs (e.g., lighting or seasonal changes)

Continuous video capture is used instead of individual photos to obtain dense temporal sampling, enabling frame-by-frame inference and higher spatial sampling density along the flight trajectory.

## 2.3 Video Link Validation

Validation of video links is critical for determining the usability of the captured footage for analysis purposes. This includes verification that the files are valid, that the timestamps are consistent, and that all necessary metadata is available. Since the videos are primarily intended to serve as the input data for vegetation analysis, the integrity of the videos is of paramount importance[8]. Recording and Exporting Video and File Formats The DJI Mini 4 Pro captures video in MP4 format and generates SRT (SubRipText) subtitle files. The SRT files typically include telemetry information, such as, timestamp, altitude, speed and camera setting. In addition, GPS coordinates are frequently included in these files in some DJI models, making them very useful for georeferencing. This validation checks that:

- MP4 video files are intact and playable
- Each video has its corresponding SRT file (when generated by the platform)
- Timestamps in the SRT are consistent and can be aligned with the video timeline

For mapping, an additional validation step verifies whether the SRT contains the specific telemetry fields required for georeferencing (in particular, latitude and longitude). This is an important practical detail: depending on device settings and export configuration, some SRT files may include only partial telemetry (e.g., timestamp and

altitude), while others contain full geodetic coordinates. Therefore, the pipeline treats "SRT with GPS" as a dataset property that must be confirmed per flight.

## 2.4 Laboratory Testing and Real-Time Considerations

A laboratory phase was conducted to assess whether the collected data could support *near-real-time* visualization and frame-level georeferencing. From a systems standpoint, real-time operation requires two conditions: (i) a reliable live video transport mechanism, and (ii) availability of per-frame (or high-rate) geospatial telemetry that can be synchronized with the stream.

While DJI provides a low-latency live preview to the controller/app, exporting that stream to external systems can require additional infrastructure (e.g., RTMP servers). RTMP-based streaming was considered as a potential integration path using an Nginx (or mediamtx) server deployed via Docker. However, even if video streaming is feasible, *live georeferenced mapping still depends on the availability and accessibility of synchronized GPS telemetry*. In practice, this introduces limitations: the video stream and telemetry may not be exposed to third-party systems at the same fidelity as offline files, and controller network isolation can prevent direct streaming to a local server [1].

As a result, the current implementation prioritizes **offline processing**, where the full-resolution MP4 and the SRT telemetry (when available) are processed post-flight to generate health maps.

## 2.5 Interim Conclusion

The DJI Mini 4 Pro provides sufficient imaging and navigation capability for structured vegetation data collection. The project workflow is robust in **post-flight mode**: recorded MP4 videos enable vegetation analysis, and accompanying SRT telemetry can be parsed into CSV to support georeferenced mapping when the required fields are present. Real-time vegetation mapping remains challenging due to the combined requirements of live video transport and synchronized, accessible geospatial telemetry; therefore, live operation is treated as an exploratory extension rather than the baseline system capability.

## 3 Live Streaming and Backend (Video Ingest Prototype)

Using the DJI Mobile SDK, developers can also obtain either the raw or decoded video stream coming from a drone on a mobile device (either Android or iOS) that is connected to the remote control unit. However, the primary focus of the DJI Mobile SDK is for use case development of local preview and/or local recording capabilities [1]. For real time streaming of the video feed over the internet, additional design choices are required for encoding, transport protocol(s), and server infrastructure to extend the original pipeline[3]. In the coursework project described above, the Remote (or mobile device) is used as the RTMP Publisher. The live video feed received from the DJI SDK is sent as RTMP packets to an RTMP server implemented using Nginx with the RTMP module. The RTMP server then distributes the received video stream to one or more viewers or to other downstream service(s) [2,3]. Advantages:

- Well established streaming protocol.

- Support for multi-client applications.
- Centralized video stream management.
- Predictable latency characteristics.
- Easy to integrate with main analytics application.

Limitations:

- Requires server infrastructure to be available externally from the Internet.
- More complex to configure than some of the other solutions.
- Centralized video stream management.

The Live Stream System Architecture includes the following components: DJI Mavic Drone – Captured live video feed. Remote / DJI Android Application – Received the video feed from the DJI SDK, encoded it, and published it using RTMP. Nginx RTMP Server – Served as the central point of video streaming, where the RTMP feed from the Remote / DJI Android Application was received and distributed to all the different client applications[2]. Client Application – A python RTMP client application that connects to the RTMP server using PyAV, and utilizes OpenCV to access the individual frames captured by the DJI Mavic Drone. The Client Application was integrated into the main analytics application. This architecture decouples video capture from video distribution and therefore provides greater flexibility in terms of scalability and analyzing latency[1].

### 3.1 RTMP Server and Client implementation

The system's core infrastructure is built around an Nginx server, utilizing the RTMP (Real-Time Messaging Protocol) module, as the primary streaming hub. Nginx was chosen due to its lightweight structure, high throughput performance when handling many simultaneous connections, and large-scale adoption in industry-level production streaming architectures. Through the RTMP module, the Nginx server accepts incoming live video feeds, manages how they are buffered, and redistributes the feed to multiple receiving devices with relatively little overhead. The Nginx server can be configured to optimize the tradeoff between latency and overall stability of the video stream through configuration options like chunk sizes, buffer lengths, and application context. In contrast to production grade streaming platforms, Nginx provides the developer full visibility into the entire streaming process, thus it is particularly well suited to experimental and educational settings. FFmpeg is used within the encoding layer, where it takes the raw video frames produced by the DJI Mobile SDK on an Android device and encodes the frames to produce a video feed compressed to H.264, the format required for RTMP transmission. FFmpeg acts as the low-level multimedia processing engine in the overall system and thus provides the ability to efficiently and programmatically configure the compression process. The extent of the codec support offered by FFmpeg, along with the amount of detail available regarding each parameter used during compression, provides a developer with sufficient flexibility to adjust the frame rate, bitrate, and frequency of keyframes, all of which directly affect the end-to-end latency of the system[7]. To enable higher-level video manipulation and to integrate custom video processing logic into the existing system, PyAV is used as a Pythonic interface to FFmpeg's lower-level libraries. PyAV greatly reduces the complexity of accessing individual video frames and enables developers to implement various additional features such as

injecting timestamps into the stream, inspecting individual frames, or selectively processing individual frames without having to interact directly with the lower-level APIs of FFmpeg. By abstracting away much of the lower level complexities of FFmpeg, PyAV enables the developer to rapidly prototype and experiment with various configurations of the system while still taking advantage of the performance characteristics of FFmpeg. After the video has been encoded, it is encapsulated as per the RTMP protocol and transmitted over a persistent TCP connection to the Nginx server. Due to the nature of RTMP's chunk based transmission paradigm, it is possible to reliably transmit video data over unstable networks while controlling the amount of buffering that occurs at the transmitting device and the receiving server. Although RTMP will never provide ultra-low latency transmission, its predictable buffering paradigm makes it suitable for measuring controlled amounts of latency in addition to providing stable live streaming. The Nginx server, acting as the central node for the RTMP server, forwards the incoming RTMP video feed to one or more subscribing clients or downstream services. As a result of the decoupling of video ingestion and video distribution, it is now possible for multiple viewing clients to concurrently receive a copy of the video feed without placing increased processing loads on the original transmitting Android device. Additionally, since the video distribution functionality exists as part of the server, it is also possible to add additional services in the future including recording, transcoding, or converting protocols[9]. In conclusion, the use of Nginx, FFmpeg, and PyAV enables a scalable and extensible streaming architecture that facilitates the real-time transmission of video from UAVs while providing the needed control and observability to perform latency evaluations driven by research.

### 3.2 Latency Profiling

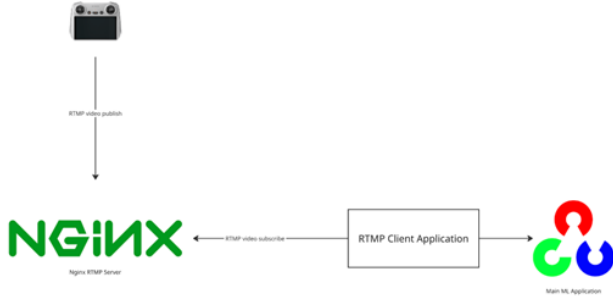
To assess whether the developed system would be suitable for real-time monitoring, a "glass-to-glass" latency profiling methodology was applied. Latency is defined here as the duration of time between the occurrence of an actual event in the field of view of the drone camera and the moment that the event is visualized on the client application screen. Photographic timestamp verification was used as the methodology to measure latency. The drone camera was focused upon a high precision stopwatch located in the field of view of the camera. The stopwatch was then visually observed on the same monitor as the client application, which was displaying the received video stream from the drone. A high speed camera was used to capture both the stopwatch and the client display simultaneously. The difference between the two timestamp values represents the total end-to-end latency of the system. Component-wise Latency Breakdown The ingest pipeline follows a server-client architecture (Figure 3):

- (1) **Acquisition and Local Transmission (Drone → Mobile):** The DJI Mini 4 Pro's O4 transmission system transmits the feed from the drone to the remote controller / mobile device. The time spent in this step is very short, ranging from approximately 150–200 milliseconds.
- (2) **Encoding RTMP Push (Mobile → Nginx Server):** The Android application needs to first decode the DJI stream before re-encoding it into an RTMP-compatible H.264 format for transmission. The combination of the decoding and network



upload to 4G/LTE introduces a variable delay of 500–800 ms, and is the least consistent of the four steps due to fluctuations in network conditions.

- (3) **Server Buffering (Nginx RTMP):** To ensure a continuous video stream (and prevent jitter), the Nginx server uses a data buffer to store incoming frames. However, the buffer stores frames for 1000–1500 ms before sending them to the client, which reduces jitter but increases the likelihood of dropped frames if the buffer size is reduced.
- (4) **Client Decoding (Nginx → Python Visualization):** The client-side PyAV and OpenCV pipeline buffers a small number of frames to provide a smooth experience for the user, adding approximately 200–300 ms to the overall latency.
- (5) **Fusion (Health ↔ Coordinates):** health estimates are synchronized with telemetry using timestamps and/or frame indices, generating the enriched dataset used to produce flight maps and analytical heat/risk maps.



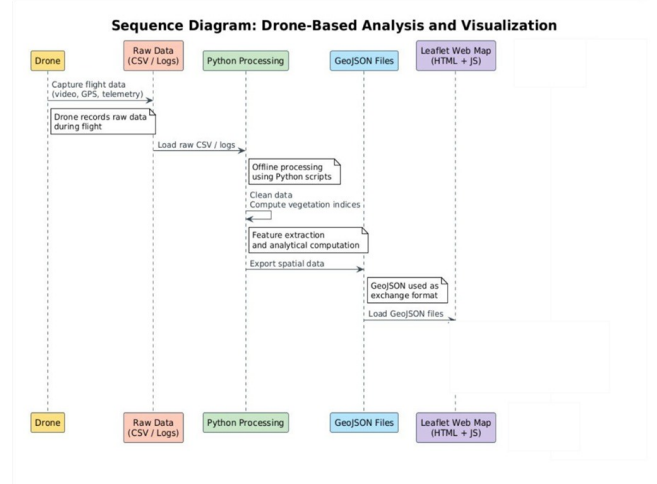
**Figure 1: Process architecture for video ingest and deferred georeferencing.**

An additional challenge faced during the profiling effort was the synchronization of the video stream with telemetry data. Video data is delivered via RTMP with a latency of approximately 3 seconds, whereas telemetry data (e.g., GPS coordinates delivered via separate API calls) arrive nearly instantly (<500 ms). This creates a "ghosting" effect in the live interface: the drone marker on the map appears to move ahead of the video footage[5]. To address this challenge for the purpose of this thesis, we determined that for live mapping to accurately correlate video with geography, a circular buffer must be created on the client-side to delay the telemetry data signal by  $t_{\text{latency}}$  so that the location of the map marker aligns perfectly with the video frame currently being processed

## 4 Visualization and User Interface

To enable fast interpretation of the computed vegetation health indicators, we developed an interactive, map-based visualization as a **client-side web application**. The user interface is implemented with the **Leaflet** JavaScript library [1] and exported as a portable HTML artifact, which can be opened locally (no server required) and shared across devices.

The visualization layer serves two complementary purposes: (i) to *inspect the flight geometry* and sampling density, and (ii) to *communicate spatial patterns of vegetation stress* through a continuous heat/risk surface[2].



**Figure 2: System architecture.**

### 4.1 Map Layers and Spatial Interpolation

The analytical map overlays vegetation health on top of standard basemaps (e.g., satellite imagery or street maps). Since the drone produces measurements as **discrete samples along the flight path**, a direct point plot can be visually sparse and sensitive to sampling density. Therefore, we generate a continuous surface using **Inverse Distance Weighting (IDW)**, which provides a simple and robust interpolator without strong distributional assumptions.

Given samples  $\{(x_i, z_i)\}_{i=1}^N$ , where  $z_i \in [0, 1]$  is the health index associated with the georeferenced position  $x_i$ , the estimated health at a grid location  $x$  is:

$$z(x) = \frac{\sum_{i=1}^N w_i(x) z_i}{\sum_{i=1}^N w_i(x)}, \quad w_i(x) = d(x, x_i)^{-p}. \quad (1)$$

Here,  $d(x, x_i)$  is the planar distance between the grid cell and sample  $i$ , and  $p$  controls how quickly influence decays with distance. We use  $p = 2$  as a practical compromise: it produces smooth transitions while still preserving local anomalies when sampling density is sufficient.

*Interpretation and Limitations.* IDW is well-suited for producing an intuitive heatmap from sparse samples, but it does not model physical processes (e.g., crop growth dynamics) and can introduce artifacts when points are unevenly distributed. For this reason, we also provide the raw flight track and point layers to help users contextualize the interpolation results[4].

### 4.2 Color Mapping and Visual Semantics

To represent the normalized health index  $z \in [0, 1]$ , the interface uses a perceptually intuitive *traffic-light* color scheme: **red** indicates low health, **green** indicates high health, and **yellow/orange** represents intermediate values. Internally, the mapping is implemented as a

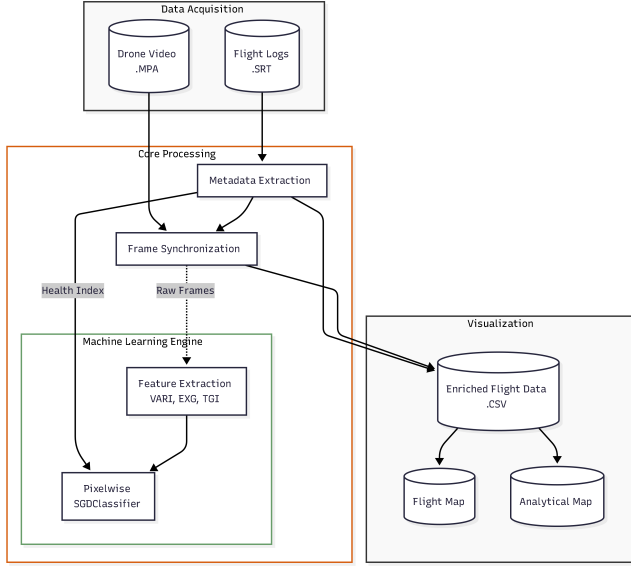


Figure 3: Sequence diagram.

continuous gradient (e.g., HSL hue interpolation from  $0^\circ$  to  $120^\circ$ ), ensuring smooth color transitions across the heatmap.

This choice supports rapid visual scanning: users can quickly identify stressed regions (warm colors) and prioritize them for inspection or further analysis.

### 4.3 Interaction Design and Custom Styling

The interface prioritizes readability and exploratory analysis:

- **Layer control:** users can toggle between key layers such as the raw flight track (polyline), sample points, and the interpolated heatmap, enabling transparent interpretation of the underlying data.
- **Legend and statistics panels:** floating UI components summarize the meaning of the color scale and provide basic descriptive information (e.g., range of health values) to support quick diagnosis.
- **CSS customization:** custom styles were applied to improve usability and visual hierarchy. For example, floating panels use high z-index to remain accessible above map tiles, and semantic color variables are used to ensure consistent styling across markers, legends, and overlays.

Overall, the visualization module bridges the gap between numeric outputs (CSV health estimates) and operational insight, providing an interpretable geospatial interface for analyzing vegetation health patterns.

## 5 Feature Extraction and Machine Learning

### 5.1 RGB Vegetation Indices

Since the DJI Mini 4 Pro provides a standard RGB camera (no NIR band), vegetation health must be inferred from visible-spectrum cues. We therefore compute a compact set of **RGB-based vegetation indices** that emphasize greenness and reduce background influence.

For each pixel with channels  $(R, G, B)$ , we extract the following features:

- **VARI** (Visible Atmospherically Resistant Index), which enhances vegetation responses while being relatively less sensitive to illumination changes:

$$\text{VARI} = \frac{G - R}{G + R - B + \epsilon} \quad (2)$$

where  $\epsilon$  is a small constant to avoid division by zero.

- **ExG** (Excess Green), which amplifies the green channel and helps separate foliage from soil or man-made surfaces:

$$\text{ExG} = 2G - R - B \quad (3)$$

- **TGI** (Triangular Greenness Index), a visible-spectrum proxy related to vegetation “greenness” patterns:

$$\text{TGI} = -0.5 [190(R - G) - 120(R - B)] \quad (4)$$

**Vegetation masking.** To reduce false detections on non-vegetated regions (e.g., bare soil, roads, shadows) [1], we apply a simple pre-filter: pixels below an ExG threshold are masked out and excluded from training and inference. This step improves robustness by restricting the classifier to candidate vegetation pixels only.

### 5.2 Pixelwise Learning Formulation

The model operates at the **pixel level**. Each pixel is represented by a feature vector  $\mathbf{x} = [\text{VARI}, \text{ExG}, \text{TGI}]^\top$  and is classified into a binary state (*healthy* vs. *stressed/non-healthy*). For a given frame, pixel predictions are aggregated to compute a frame-level health score (e.g., percentage of pixels classified as healthy), which is later fused with the flight metadata to generate georeferenced health maps[9].

### 5.3 Hybrid Training Strategy (Supervised + Weak Labels)

A main practical challenge is domain shift: controlled datasets of leaf images differ substantially from aerial drone footage in scale, lighting, background, and viewpoint. To mitigate this, we adopt a hybrid strategy combining strong supervision with weakly labeled domain-specific samples:

- (1) **Stage 1 (Supervised pre-training):** the initial classifier is trained using the **PlantVillage** dataset [2], where image-level labels (healthy vs. diseased) provide reliable supervision. This stage establishes a baseline decision boundary in the index space.
- (2) **Stage 2 (Weak-label adaptation with drone frames):** the model is incrementally updated using frames sampled from drone videos. Frames judged as visually healthy contribute positive vegetation pixels, while mixed/stressed frames are handled with conservative heuristics (e.g., thresholding-based weak masks) to generate approximate labels. This step adapts the model to field-specific illumination and background conditions without requiring dense manual annotation.

### 5.4 Incremental Classifier and Scalability

We use an **SGDClassifier** configured for logistic regression (`log_loss`) with `partial_fit`. This choice is driven by scalability: training can be performed on millions of pixels in a streaming fashion, without

loading the full dataset into memory. The model learns a linear decision function:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (5)$$

where  $\mathbf{w} = [w_{\text{VARI}}, w_{\text{EXG}}, w_{\text{TGI}}]^\top$  and  $b$  are learned parameters and  $\sigma(\cdot)$  is the logistic function. This formulation is computationally efficient and stable for iterative updates.

## 5.5 Interpretability and Visual Evidence

A linear pixelwise model provides intrinsic interpretability, which is valuable for agronomic validation and debugging:

- **Global interpretability:** the sign and magnitude of coefficients  $\mathbf{w}$  indicate how each index contributes to the healthy vs. stressed decision.
- **Local interpretability:** pixel predictions can be rendered as semi-transparent masks over the original frames, directly highlighting which regions drive the final health score.

## 6 Evaluation

This section evaluates the proposed drone-based vegetation health detection system using a combined framework of **quantitative metrics** and **qualitative error inspection**. The objective is to measure how reliably the system separates *healthy vegetation* (dense, vigorous canopy) from *stressed / potentially hazardous vegetation* (dry, senescent, or visually degraded biomass) under realistic RGB-only constraints. Since the application is intended to support operational decision-making (e.g., prioritizing inspection zones), we emphasize the trade-off between missed hazards and false alarms.

### 6.1 Evaluation Metrics

We report four standard metrics for binary classification. These metrics are chosen because the relative cost of errors is asymmetric: **false negatives** (missing a hazardous patch) are typically more costly than **false positives** (raising an unnecessary warning), especially when the system is used as a screening tool[6].

- **Accuracy:** proportion of correct predictions over all samples. Although useful as a baseline, accuracy may hide class imbalance effects and is therefore interpreted together with class-dependent metrics.
- **Precision (Positive Predictive Value):** reliability of the *dangerous* prediction. It answers: *When the model flags a sample as hazardous, how often is it correct?* High precision reduces operator burden and improves trust in alerts.
- **Recall (Sensitivity):** coverage of the *dangerous* class. It answers: *Of all truly hazardous samples, how many were detected?* High recall is critical when the primary goal is to avoid overlooking risk areas.
- **F1-score:** harmonic mean of precision and recall, providing a single robustness indicator that penalizes extreme trade-offs.

### 6.2 Results and Discussion

Table 1 summarizes the quantitative performance. Overall, the system achieves a strong baseline with high precision and balanced F1, indicating that the learned vegetation-index decision boundary generalizes reasonably well to unseen samples.

**Table 1: Evaluation results on the test set (Drone vs. Kaggle).**

| Metric    | Value           | Interpretation                                    |
|-----------|-----------------|---------------------------------------------------|
| Accuracy  | 0.8537 (Drone)  | Strong overall correctness                        |
|           | 0.8300 (Kaggle) |                                                   |
| Precision | 0.8908 (Drone)  | Alerts are mostly reliable (low false-alarm rate) |
|           | 0.8700 (Kaggle) |                                                   |
| Recall    | 0.8073 (Drone)  | Detected > 80% of hazardous cases                 |
|           | 0.9400 (Kaggle) |                                                   |
| F1-score  | 0.8470 (Drone)  | Balanced precision–recall trade-off               |
|           | 0.9000 (Kaggle) |                                                   |

From an operational perspective, the high precision (0.89) is valuable because it keeps the alert stream actionable: when the system marks vegetation as stressed, the probability of a true positive is high. The recall (0.81) shows that most hazardous cases are detected, but also indicates that a non-negligible portion remains missed. In practical deployments, this gap can often be addressed by tuning the decision threshold to prioritize recall (pessimistic detection) when the cost of missed hazards is higher than the cost of extra inspections.

### 6.3 Qualitative Error Analysis and Limitations

A qualitative inspection of failure cases reveals recurring patterns that explain the remaining errors and highlight directions for improvement:

- **Shadow sensitivity:** deep shadows can be misclassified as stressed vegetation because all RGB channels drop simultaneously, distorting index values and mimicking low-greenness responses.
- **Phenological ambiguity:** natural senescence (seasonal yellowing) may be confused with disease-like stress, since both produce similar visible-spectrum signatures in RGB indices.
- **Altitude dependence:** performance can degrade at higher flight altitudes where fine vegetation texture is lost and pixels represent larger ground areas, increasing mixing between canopy and background.
- **Temporal instability (frame-to-frame flicker):** pixelwise predictions may oscillate between classes due to small view-point changes, compression artifacts, or illumination fluctuations, suggesting that temporal smoothing or tracking-based aggregation could improve stability in video mode.

## 7 Robustness and Testing

This section summarizes robustness and stability testing of the drone monitoring pipeline under two operational modes: (i) a **video-ingest (live) prototype** for near-real-time frame processing, and (ii) the **post-flight (offline)** pipeline, which remains the source of truth for georeferenced mapping. The goal is to verify continuous operation under sustained input load, document expected failure modes, and ensure that final outputs remain reliable even when live ingest is unstable[3].

## 7.1 Stress Testing

Robustness was evaluated by exercising the pipeline with sustained inputs and verifying that processing completes without interruptions or corrupted outputs.

- **Live video ingest:** the client was tested for continuous stream consumption, frame decoding, and health inference under variable network conditions. The objective was to confirm that temporary transport jitter affects responsiveness but does not crash the application or break the processing loop.
- **Offline processing:** full-flight 4K recordings were processed end-to-end to validate scalability of the MP4+SRT workflow, including metadata parsing, frame synchronization, pixelwise inference, and generation of enriched CSV outputs used for mapping.

## 7.2 Observed Failure Modes

The following failure cases were identified as the most relevant for operational usage:

- **Stream unavailable / endpoint mismatch:** incorrect RTMP URLs, port exposure, or server configuration prevents the client from connecting to the live stream.
- **Missing artifacts:** offline processing requires paired inputs; if the corresponding .MP4 or .SRT file is missing (or renamed inconsistently), the fusion step cannot be completed and map generation is blocked.
- **Temporal misalignment:** live ingest does not guarantee synchronized per-frame telemetry. Even when video is available, georeferenced mapping may be inaccurate without a shared time base. This reinforces the role of the offline pipeline, where SRT telemetry is explicitly parsed and aligned to the processed frames.

## 7.3 Stability Under Load and Graceful Degradation

Overall system stability is achieved through a two-stage design with **graceful degradation**. The live mode supports immediate monitoring by producing time-indexed health estimates from incoming frames; however, it is not treated as the authoritative source for final maps. When live ingest is disrupted (e.g., stream dropouts, buffering, or timing inconsistencies), the pipeline falls back to the offline MP4+SRT workflow[2].

In offline mode, recorded video is fused with telemetry extracted from the SRT logs to produce a synchronized dataset containing geographic coordinates, frame indices (or timestamps), and computed health values. This dataset is then used to generate the final flight and analytical maps. As a result, transient live-stream instability does not compromise the reproducibility or correctness of the final georeferenced outputs.

## 8 Conclusion

This work delivered and validated an end-to-end **drone-based vegetation health mapping pipeline** that transforms consumer UAV footage into **georeferenced, interpretable health/risk maps**. Using a DJI Mini 4 Pro as the acquisition platform, the project established a practical workflow that (i) collects structured 4K data via repeatable

waypoint missions, (ii) extracts flight telemetry from DJI SRT logs, (iii) performs pixelwise vegetation classification using RGB-derived indices, and (iv) visualizes results as interactive Leaflet-based maps.

The proposed machine learning module relies on visible-spectrum features (VARI, ExG, and TGI) and an incremental SGDClassifier trained in a scalable, pixelwise manner. Quantitatively, the classifier achieved an accuracy of 85.37% and a precision of 89.08%, indicating that generated “danger” alerts are largely reliable and do not overwhelm users with false positives. Qualitatively, the visualization layer provides an operational interface: flight trajectories and interpolated heat surfaces enable users to rapidly locate stressed regions and interpret results in geographic context.

A central limitation is that **true real-time georeferenced mapping is not guaranteed in the current DJI workflow**. Although a live video-ingest prototype was explored (via RTMP with MediaMTX as the current implementation and Nginx as an evaluated alternative), the system does not have dependable access to synchronized per-frame telemetry without a DJI SDK bridge. Consequently, the authoritative mapping product remains the **offline post-flight pipeline** that fuses MP4 frames with coordinates extracted from SRT logs. In addition, end-to-end RTMP “glass-to-glass” latency on typical networks (approximately 2–3.5 seconds) reduces responsiveness for interactive maneuvering and limits the usefulness of live overlays beyond situational preview.

Error inspection further shows that challenging illumination conditions (e.g., strong shadows and abrupt lighting changes) can bias RGB indices, occasionally causing shaded healthy vegetation to be misclassified as “Dangerous”. These observations motivate two clear directions for future work. First, enabling **live telemetry access** (e.g., via DJI Mobile SDK integration) would support true online fusion between frames and position data. Second, robustness can be improved through **temporal stabilization** (to reduce frame-to-frame flicker) and **illumination/shadow normalization** to suppress environmental noise in RGB-only imagery.

Overall, the project demonstrates a feasible and reproducible academic prototype: live ingest can support near-real-time monitoring, while the offline MP4+SRT workflow guarantees correct, georeferenced health/risk maps. Addressing telemetry access and illumination robustness is the key step to move from a validated prototype toward reliable field deployment.

Finally, to complement the written results, we recorded a short field-demonstration video showing the drone operation and examples of the captured imagery under typical conditions and under snowfall. The video illustrates practical aspects of the acquisition workflow and how weather/illumination changes can affect RGB-only mapping results [3]. Watch it here: [YouTube video link](#).

## References

- [1] Vladimir Agafonkin. 2024. Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps. Project website. <https://leafletjs.com/> Accessed: 2026-01-30.
- [2] David P. Hughes and Marcel Salathé. 2015. PlantVillage Dataset. TensorFlow Datasets catalog entry. [https://www.tensorflow.org/datasets/catalog/plant\\_village](https://www.tensorflow.org/datasets/catalog/plant_village) Accessed: 2026-01-30.
- [3] YouTube. 2026. DRONE. YouTube video. <https://www.youtube.com/watch?v=JICub7y1Xnw> Accessed: 2026-01-30.